Universidad Nacional del Comahue
Facultad de Informática

Trabajo Práctico Obligatorio
Diseño de Compiladores e Interpretes

Quintero, Kevin	FAI-2771
Rodriguez, Jeremías	FAI-2543
Sepulveda, Luciano	Leg. 72662

Neuquén Argentina

2026

Índice

1. Descripción del problema **3**

2. Descripción de la gramática **3**

2.1. Notación utilizada . . . 3
2.2. Estructura general del programa . . . 3
2.3. Declaraciones de variables . . . 3
2.4. Constantes predefinidas . . . 4
2.5. Declaraciones de subprogramas . . . 4
2.6. Sentencias . . . 4
2.7. Entrada y salida . . . 5
2.8. Expresiones . . . 5
2.8.1. Expresión general . . . 5
2.8.2. Expresiones aritméticas y booleanas . . . 5
2.8.3. Variables y llamadas a función . . . 5
2.8.4. Operadores relacionales . . . 6
2.9. Números e identificadores . . . 6

3. Problemas encontrados **6**

4. Conclusiones **6**

Bibliografía **7**

5. Alfabeto **7**

6. Lexemas **8**

7. Expresiones regulares **9**

1. Descripción del problema

1. Definir en BNF extendido el subconjunto del lenguaje Pascal integrado por los siguientes elementos:
   - definición de variables y subprogramas.
   - tipos de datos simples: enteros y booleanos
   - constantes definidas por el lenguaje: true, false
   - subprogramas: procedimientos y funciones, solamente con pasaje de parámetros por valor.
   - sentencias: de asignación, alternativa (if then else), repetitiva (while) y sen tencia compuesta (begin end).
   - procedimientos de entrada/salida para un solo valor de tipo entero (read y write).
   - expresiones aritméticas con los operadores $+$, $-$ (unario y binario), $*$, / y los paréntesis (,).
   - expresiones booleanas con los operadores AND, OR y NOT y los operadores de comparación $>$, $<$, $=$, $<=$, $>=$ y $<>$ entre expresiones aritméticas. Operadores de comparación entre $=$ y $<>$ para expresiones booleanas.

2. Descripción de la gramática

2.1. Notación utilizada

La gramática se expresa en **Notación BNF Extendida** (EBNF), utilizando las siguientes convenciones:

- Los **no-terminales** se escriben entre paréntesis angulares en cursiva: ⟨*ejemplo*⟩.
- Los **terminales** se escriben en tipografía monoespaciada: `begin`.
- Los corchetes $[\,\alpha\,]$ indican que α es **opcional** (cero o una ocurrencia).
- Las llaves $\{\,\alpha\,\}$ indican **repetición** (cero o más ocurrencias).
- La barra vertical | indica **alternativa**.

2.2. Estructura general del programa

Un programa en este subconjunto de Pascal se compone de un encabezado seguido de un bloque principal. El bloque agrupa las declaraciones de variables, las declaraciones de subprogramas y el cuerpo ejecutable.

⟨*programa*⟩ ::= `program` ⟨*identificador*⟩ `;` ⟨*bloque*⟩ `.`

⟨*bloque*⟩ ::= [⟨*parte de declaraciones de variables*⟩]
[⟨*parte de declaraciones de subrutinas*⟩]
⟨*comando compuesto*⟩

2.3. Declaraciones de variables

Las variables deben declararse antes de ser utilizadas, especificando su tipo. Los únicos tipos de datos admitidos son `integer` y `boolean`. Es posible declarar múltiples grupos de variables en una misma sección, separados por punto y coma.

⟨*parte de declaraciones de variables*⟩	::=	`var` ⟨*declaración de variables*⟩ { `;` ⟨*declaración de variables*⟩ } `;`
⟨*declaración de variables*⟩	::=	⟨*lista de identificadores*⟩ `:` ⟨*tipo*⟩
⟨*lista de identificadores*⟩	::=	⟨*identificador*⟩ { `,` ⟨*identificador*⟩ }
⟨*tipo*⟩	::=	`integer` | `boolean`

2.4. Constantes predefinidas

El lenguaje define dos constantes booleanas predeclaradas: `true` y `false`. Estas constantes no pueden ser redefinidas por el programador y son consideradas identificadores especiales del lenguaje.

⟨*booleano*⟩ ::= `true` | `false`

2.5. Declaraciones de subprogramas

Se admiten dos tipos de subprogramas: **procedimientos**, que no retornan valor, y **funciones**, que retornan un valor de tipo simple. El pasaje de parámetros es **exclusivamente por valor**; no se admite el modificador `var` en las secciones de parámetros formales, lo que implica que los argumentos son copiados al momento de la llamada y los cambios internos no afectan a las variables del programa invocante.

⟨*parte de declaraciones de subrutinas*⟩	::=	{ ⟨*declaración de procedimiento*⟩ `;` } { ⟨*declaración de función*⟩ `;` }
⟨*declaración de procedimiento*⟩	::=	`procedure` ⟨*identificador*⟩ [⟨*parámetros formales*⟩] `;` ⟨*bloque*⟩
⟨*declaración de función*⟩	::=	`function` ⟨*identificador*⟩ [⟨*parámetros formales*⟩] `:` ⟨*tipo*⟩ `;` ⟨*bloque*⟩
⟨*parámetros formales*⟩	::=	`(` ⟨*sección de parámetros formales*⟩ { `;` ⟨*sección de parámetros formales*⟩ } `)`
⟨*sección de parámetros formales*⟩	::=	⟨*lista de identificadores*⟩ `:` ⟨*tipo*⟩

2.6. Sentencias

Las sentencias admitidas son: asignación, llamada a procedimiento, sentencia compuesta, condicional, repetitiva y entrada/salida. La sentencia condicional presenta una ambigüedad inherente en el `else` opcional, que se resuelve asociando cada `else` al `if` más próximo que no haya sido cerrado.

⟨*comando compuesto*⟩	::=	`begin` ⟨*comando*⟩ { `;` ⟨*comando*⟩ } [`;`] `end`
⟨*comando*⟩	::=	⟨*asignación*⟩ | ⟨*llamada de procedimiento*⟩ | ⟨*comando compuesto*⟩ | ⟨*comando condicional*⟩ | ⟨*comando repetitivo*⟩ | ⟨*comando entrada-salida*⟩
⟨*asignación*⟩	::=	⟨*variable*⟩ `:=` ⟨*expresión*⟩
⟨*llamada de procedimiento*⟩	::=	⟨*identificador*⟩ [`(` ⟨*lista de expresiones*⟩ `)`]
⟨*comando condicional*⟩	::=	`if` ⟨*expresión*⟩ `then` ⟨*comando sin rótulo*⟩ [`else` ⟨*comando*⟩]
⟨*comando repetitivo*⟩	::=	`while` ⟨*expresión*⟩ `do` ⟨*comando*⟩

2.7. Entrada y salida

Los procedimientos de entrada/salida operan sobre **un único valor de tipo entero**. El procedimiento `read` lee un valor desde la entrada estándar y lo almacena en la variable indicada. El procedimiento `write` imprime el valor de una variable entera en la salida estándar. No se admite la lectura o escritura de valores booleanos ni de múltiples valores en una misma llamada.

$$\langle \textit{comando entrada-salida} \rangle \quad ::= \quad \texttt{read} \ \texttt{(} \ \langle \textit{identificador} \rangle \ \texttt{)} \ | \ \texttt{write} \ \texttt{(} \ \langle \textit{variable} \rangle \ \texttt{)}$$

2.8. Expresiones

Las expresiones se dividen en **aritméticas** y **booleanas**. La gramática refleja la precedencia de operadores mediante la jerarquía de producciones: los operadores de menor precedencia aparecen en los niveles superiores, y los de mayor precedencia en los niveles inferiores. Esto garantiza que la estructura del árbol de derivación refleje correctamente el orden de evaluación.

2.8.1. Expresión general

Una expresión consiste en una expresión simple, opcionalmente seguida de un operador relacional y otra expresión simple. Esto permite tanto expresiones puramente aritméticas como comparaciones que producen un resultado booleano.

$$\begin{array}{lcl}
\langle \textit{expresión} \rangle & ::= & \langle \textit{expresión simple} \rangle \ [\, \langle \textit{relación} \rangle \ \langle \textit{expresión simple} \rangle \,] \\
\langle \textit{lista de expresiones} \rangle & ::= & \langle \textit{expresión} \rangle \ \{\, \texttt{,} \ \langle \textit{expresión} \rangle \,\}
\end{array}$$

2.8.2. Expresiones aritméticas y booleanas

Los operadores aritméticos admitidos son $+$ y $-$ en forma binaria (suma y resta) y unaria (positivo y negativo), junto con $*$ (multiplicación) y $/$ (división). Los operadores booleanos `or`, `and` y `not` se integran en la misma jerarquía: `or` tiene la menor precedencia, seguido de `and`, y finalmente `not` que opera sobre un único factor.

$$\begin{array}{lcl}
\langle \textit{expresión simple} \rangle & ::= & [\,\texttt{+} \ | \ \texttt{-}\,] \ \langle \textit{término} \rangle \ \{\, (\texttt{+} \ | \ \texttt{-} \ | \ \texttt{or}) \ \langle \textit{término} \rangle \,\} \\
\langle \textit{término} \rangle & ::= & \langle \textit{factor} \rangle \ \{\, (\texttt{*} \ | \ \texttt{div} \ | \ \texttt{and}) \ \langle \textit{factor} \rangle \,\} \\
\langle \textit{factor} \rangle & ::= & \langle \textit{variable} \rangle \ | \ \langle \textit{número} \rangle \ | \ \langle \textit{llamada de función} \rangle \\
 & & | \ \texttt{(} \ \langle \textit{expresión} \rangle \ \texttt{)} \ | \ \texttt{not} \ \langle \textit{factor} \rangle
\end{array}$$

2.8.3. Variables y llamadas a función

$$\begin{array}{lcl}
\langle \textit{variable} \rangle & ::= & \langle \textit{identificador} \rangle \ | \ \langle \textit{identificador} \rangle \ [\, \langle \textit{lista de expresiones} \rangle \,] \\
\langle \textit{llamada de función} \rangle & ::= & \langle \textit{identificador} \rangle \ [\, \texttt{(} \ \langle \textit{lista de expresiones} \rangle \ \texttt{)} \,]
\end{array}$$

2.8.4. Operadores relacionales

Los operadores de comparación entre expresiones aritméticas son seis: igualdad, desigualdad, menor, menor o igual, mayor o igual y mayor. Entre expresiones booleanas solo se admiten la igualdad (=) y la desigualdad (≠). Esta restricción evita comparaciones sin semántica definida entre valores lógicos.

$$\langle \textit{relación} \rangle \quad ::= \quad \texttt{=} \mid \texttt{<>} \mid \texttt{<} \mid \texttt{<=} \mid \texttt{>=} \mid \texttt{>}$$

2.9. Números e identificadores

Los identificadores comienzan con una letra y pueden contener letras y dígitos. Los números son secuencias de uno o más dígitos decimales. En la práctica, su reconocimiento se delega al analizador léxico, que los trata como unidades atómicas (tokens).

$$\begin{array}{lcl} \langle \textit{número} \rangle & ::= & \langle \textit{dígito} \rangle \ \{ \ \langle \textit{dígito} \rangle \ \} \\ \langle \textit{identificador} \rangle & ::= & \langle \textit{letra} \rangle \ \{ \ \langle \textit{letra} \rangle \ \mid \ \langle \textit{dígito} \rangle \ \} \end{array}$$

3. Problemas encontrados

El principal inconveniente durante el desarrollo de este trabajo fue el poco conocimiento previo del lenguaje Pascal, dado que no es un lenguaje que usemos regularmente. Esta falta de familiaridad con el lenguaje dificultó la comprensión de ciertas construcciones sintácticas y obligó a consultar bibliografía adicional para definir correctamente el subconjunto requerido.

Adicionalmente, la gramática definida no contemplaba explícitamente el manejo de espacios en blanco entre los tokens del lenguaje. Esto generó dificultades al momento de verificar la correctitud de los programas generados, ya que herramientas de validación como BNF Playground trataban las secuencias de caracteres de forma literal, sin ignorar automáticamente los espacios. Como consecuencia, fue necesario extender la gramática para incluir un símbolo no-terminal dedicado al espacio en blanco, lo que añadió complejidad adicional a la definición original.

4. Conclusiones

El desarrollo de este trabajo se centró en la construcción de una gramática que represente de manera adecuada un conjunto reducido de elementos del lenguaje pascal, partiendo de las definiciones propuestas por Kowaltowski [1]. Durante el proceso, se evidenció que la definición de una gramática no es una tarea meramente mecánica, sino que requiere un análisis constante sobre cómo modelar correctamente cada estructura del lenguaje.

Gracias a la simplicidad del lenguaje propuesto, se obtuvo una gramática más clara, estructurada y fácil de manipular, aunque con una cobertura limitada respecto al lenguaje original. Esta decisión facilitó tanto la comprensión como la implementación de la gramática en un contexto académico. En este sentido, se logró un modelo más accesible para el análisis sintáctico y el estudio de sus componentes principales.

En conclusión, el trabajo permitió no solo adaptar una gramática existente, sino también reflexionar sobre las decisiones de diseño involucradas, destacando el impacto que tiene el equilibrio entre simplicidad y expresividad en la construcción de lenguajes dentro del ámbito de los compiladores e intérpretes.

Bibliografía

[1] T. Kowaltowski. *Implementação de Linguagens de Programação.* Guanabara Dois, 1983.

5. Alfabeto

— letras: a,b,c,...,z — dígitos: 0,1,2,...,9 — símbolos especiais: sd), S.>D +] — símbolos especiais compostos: :=,..(4,— símbolos compostos em negrito: program, label, type, array, of, var, procedure, function, begin, end, if, then, else, while, do, or, and, div, not.

6. Lexemas

Lexema (palabra)	Token	Valor (Atributo)
Cualquier identificador	ID	Puntero a la entrada en TS
Cualquier Numero	NUMERO	Puntero a la entrada en TS
program	PROGRAM	-
var	VAR	-
integer	INTEGER	-
boolean	BOOLEAN	-
procedure	PROCEDURE	-
function	FUNCTION	-
begin	BEGIN	-
end	END	-
if	IF	-
then	THEN	-
else	ELSE	-
while	WHILE	-
do	DO	-
read	READ	-
write	WRITE	-
true	VALOR_BOOLEANO	TRUE
false	VALOR_BOOLEANO	FALSE
:=	ASIGNACION	-
+	OPERADOR_MATEMATICO	SUMA
-	OPERADOR_MATEMATICO	RESTA
*	OPERADOR_MATEMATICO	MULTIPLICACION
div	OPERADOR_MATEMATICO	DIVISION_ENTERA
or	OPER_LOGICO	OR
and	OPER_LOGICO	AND
not	OPER_LOGICO	NOT
=	OPER_RELACIONAL	IGUAL
<>	OPER_RELACIONAL	DISTINTO
<	OPER_RELACIONAL	MENOR
<=	OPER_RELACIONAL	MENOR_IGUAL
>	OPER_RELACIONAL	MAYOR
>=	OPER_RELACIONAL	MAYOR_IGUAL
;	FIN_SENTENCIA	-
.	FIN_PROGRAMA	-
:	DECLARACION_TIPO	-
,	SEPARADOR	-
(	PARENTESIS_IZQ	-
)	PARENTESIS_DER	-

7. Expresiones regulares

Token	**Patrón**	**Lexema de ejemplo**
program	program	program
id	$(a-z \cup A-Z)(a-z \cup A-Z \cup 0-9)^*$	holaMundo10
var	var	var
asignacion_tipo	:	:
asignacion	:=	:=
punto_coma	;	;
coma	,	,
punto	.	.
parentesis_izq	(	(
parentesis_der	)	)
procedure	procedure	procedure
function	function	function
integer	integer	integer
boolean	boolean	boolean
begin	begin	begin
end	end	end
if	if	if
then	then	then
else	else	else
while	while	while
do	do	do
oper_logico	OR | AND | NOT	OR
numero	$(0-9)^+$	43
oper_aritmetico	+ | - | * | /	+
oper_relacional	= | < | <= | > | >= | <>	<
	$\{(\Sigma - \{\})^*\}$	{esto es un comentario}